

22. Structures: defining your own types

Herman Kamper

Every variable we have declared so far has had one of a handful of built-in types: `int`, `double`, `char`, an array of one of those, or a pointer to one of those. This lecture introduces a way to define your own types, built out of the types you already have. The keyword that does this is `struct`, and it turns out to be the one of the last new keywords you learn in this course. Everything from here on is about combining things you already know. (There is one other keyword, `typedef`, that you also learn in this lecture.)

A `struct` groups a handful of related variables together and lets you treat them as a single unit. That sounds abstract, so the whole of this lecture is really just one long example, worked from several directions, of why that turns out to matter.

A motivating example

Suppose you are writing a small program that does some applied mathematics: adding vectors together, say, and taking cross products. A vector in three dimensions has three components, which physicists and engineers usually write i , j and k .

Without structures, each vector needs three separate variables:

```
double force1i = 0.5;
double force1j = 5;
double force1k = -0.5;

double position1i = 2;
double position1j = 3;
double position1k = 5.5;

double position2i = 1.5;
double position2j = -2.3;
double position2k = 5;

double position3i = 0.5;
double position3j = 5;
double position3k = -2.3;
```

Four vectors, twelve variables. They all share the same shape: three doubles that belong together. Now suppose you want to write a function that adds up the three position vectors and crosses the

result with the force vector. Every one of those twelve variables has to be passed in separately:

```
addPositionVectorsAndCrossWithForce(force1i, force1j, force1k,  
                                     position1i, position1j, position1k,  
                                     position2i, position2j, position2k,  
                                     position3i, position3j, position3k);
```

That is a painfully long line, and it is easy to get the order of the arguments wrong without the compiler ever telling you. What we actually want is a new type, call it `vector`, so that a single variable `force1` carries its i , j and k together, and the call above becomes:

```
addPositionVectorsAndCrossWithForce(force1, position1, position2, position3);
```

That is the entire point of structures: instead of three separate doubles that you have to keep track of by hand, you define one variable that has an i , a j , and a k inside it. This is a silly example on its own. But almost every non-trivial program is solving a problem that is made up of smaller, related pieces, and structures are how you give a name to one of those pieces.

Defining a struct

Here is how you define your own type. You write the keyword `struct`, followed by a name for the new type, and then, inside curly braces, the variables that make it up. These variables are called the structure's *members*.

```
struct <name>  
{  
    <members>  
};
```

There is one thing about this syntax that everybody forgets at least once: the closing brace needs a semicolon after it. It is easy to miss, because none of the other block structures in C (`if`, `while`, function bodies) end that way. Forget it and the compiler will tell you, usually in a confusing way, so it is worth just training your fingers to type the semicolon the moment you type the closing brace.

For the vector example we would have:

```
struct vector  
{  
    double i;  
    double j;  
    double k;  
};
```

This line by itself does nothing to memory. It does not create any variables and it does not reserve any space. All it does is teach the compiler about a new type, the same way `int` or `double` are types, except that this one is called `struct vector` and it is made up of three doubles. You would put this definition near the top of your file, above `main`, in the same place you would put

an enum definition, and for the same reason: you are defining something that the rest of the file can then use.

Only when you actually declare a variable of this type does anything happen in memory, for instance if you have the following in main:

```
struct vector force1 = {0.5, 5, -0.5};
```

Now `force1` exists. It has a `.i`, a `.j` and a `.k`, and you reach them with the dot, as we will see below.

What this looks like in memory

Here is the line again, together with what it does to memory.

```
struct vector force1 = {0.5, 5, -0.5};
```

`struct vector force1 = {0.5, 5, -0.5};`



Figure 1: The variable `force1` in memory. The type `struct vector` says that three doubles belong together, so declaring `force1` creates three boxes side by side, initialised in the order the members were defined: `i`, then `j`, then `k`.

`force1` is not really one box. It is three boxes, sitting right next to each other in memory, each the size of a double. The name of the first box is `force1.i`, the name of the second is `force1.j`, and the name of the third is `force1.k`, and C knows to call them that because the type `struct vector` says so: it says the first member is called `i`, the second `j`, and the third `k`. The values in the braces are assigned to the members in that same order, so `0.5` goes into `.i`, `5` goes into `.j`, and `-0.5` goes into `.k`.

It is not an array, since the three boxes do not have to be the same type (we'll see that next). But it behaves a lot like an array in the sense that declaring the structure variable creates a contiguous block of boxes, and each member is just a name for one of them.

Exactly like an ordinary variable, you can assign a value to a member after the fact:

```
force1.k = 9;
```

This changes only the third box. After this line, `force1` is `{0.5, 5, 9}`.

Because the three members sit next to each other, the address of the whole structure is the same as the address of its first member:

```
printf("%p\n", &force1);  
printf("%p\n", &force1.i);
```

These two lines print the same address. Internally, C knows that `force1` is a `struct vector`, so it knows exactly how many bytes to reserve for it: one double for `i`, one for `j`, one for `k`. If you ask `sizeof` about it:

```
printf("%d\n", sizeof(force1));
```

it prints 24, which is three doubles at 8 bytes each. `sizeof` on a structure gives you the number of bytes needed for all of its members together.

More than one variable of the same type

Once `struct vector` is defined, you can declare as many variables of that type as you like, each with its own three boxes:

```
struct vector position1 = {2, 3, 5.5};  
struct vector position2 = {1.5, -2.3, 5};  
struct vector position3 = {0.5, 5, -2.3};
```

`position1`, `position2` and `position3` are three completely independent sets of boxes. Changing `position1.i` has no effect on `position2` or `position3`. Each is a `struct vector`, in exactly the sense that a variable can be an `int` or a `double`. Here the variable (the box) is a value of type `struct vector`, three doubles bundled under one name.

Structures with different kinds of members

Nothing says the members of a structure all have to be the same type. A structure can mix chars, ints, arrays, even pointers, in any combination:

```
struct student  
{  
    char name[30];  
    int age;  
    long studentNumber;  
};
```

This groups a name, an age and a student number, three different kinds of data, into a single type. If you were building a small database of students, this is exactly the kind of building block you would want: one `struct student` variable per student, instead of three separate arrays that you have to keep synchronised by hand.

A structure can even contain a pointer to another structure of the same type:

```
struct node
{
    struct node *leftPtr;
    struct node *rightPtr;
};
```

This looks alarming at first, a `struct node` that contains pointers to more `struct node`s, but it is fine, and it is in fact the basic building block for linked lists and trees, which show up later in this course. What is *not* allowed is a structure containing an actual instance of itself:

```
struct node
{
    struct node left;    /* not allowed */
};
```

Think about why: to know how big `struct node` is, the compiler would first need to know how big `struct node` is, which needs to know how big `struct node` is, forever. It is a circular definition, and the compiler correctly refuses it, with an error to the effect that the member has an incomplete type. A pointer does not have this problem, because a pointer is always the same size no matter what it points at: it is just a box holding an address. So a `struct` cannot contain itself, but it can perfectly well contain a pointer to its own type (i.e. it can store an address).

Here is one example. Suppose you wanted every student to know who their best friend in the course is:

```
struct student
{
    char name[30];
    int age;
    struct student *bestFriend;
};
```

That is allowed, because `bestFriend` is only a pointer, an address, not a whole second student folded inside the first.

Initialising structures

You have already seen the basic pattern: a list of values in braces, assigned to the members in the order they were declared.

```
struct coord
{
    double x;
    double y;
};

struct coord origin = {0.0, 0.0};
```

The same works for a structure with mixed member types:

```
struct person
{
    char firstName[20];
    char lastName[20];
    int age;
    char gender;
};

struct person DEls = {"Danie", "Els", 22, 'M'};
```

This sets `DEls.firstName` to "Danie", `DEls.lastName` to "Els", `DEls.age` to 22 and `DEls.gender` to 'M', again purely by position: first value to first member, second value to second member, and so on.

You can also declare more than one variable of a structure type on a single line:

```
struct coord lCorner, origin = {0.0, 0.0};
```

Copying and pointing to structures

A structure variable behaves like any other variable in two further ways. First, you can assign one structure variable to another of the same type, and it copies every member across in one go:

```
struct coord lCorner = {9.0, 1.0};
struct coord origin = {0.0, 0.0};
```

```
lCorner = origin;
```

After this line, `lCorner.x` is 0.0 and `lCorner.y` is 0.0, copied from `origin`. The two variables remain entirely separate: changing `origin` afterwards does not change `lCorner`.

Second, you can take the address of a structure variable with `&`, exactly as you would for an `int` or a `double`, and store that address in a pointer:

```
struct coord *cPtr;

cPtr = &lCorner;
```

`cPtr` now holds the address of `lCorner`. This is nothing new: a pointer is a pointer, and a structure variable lives at an address like everything else. The pointer's type just happens to be "pointer to struct coord" rather than "pointer to int". You can also deference a pointer to a struct with `*`, exactly as before (more of this below).

Accessing members: the dot and the arrow

You already know the dot operator: if `c` is a structure variable, `c.x` is its `x` member. The dot only works when the thing on its left is the structure itself, not a pointer to one.

If instead you have a pointer to a structure, you need to dereference the pointer before you can use the dot:

```
struct coord c;  
struct coord *cPtr;
```

```
cPtr = &c;
```

```
(*cPtr).x = 10.5;
```

The parentheses matter here. `*cPtr` has to happen before `.x`, because dereferencing binds more loosely than the dot; without the parentheses, `*cPtr.x` would try to take `cPtr.x` first, which does not typecheck, since `cPtr` is a pointer and has no member called `x`.

Because this pattern – dereference a structure pointer and then reach into a member – comes up constantly once you start passing structures around by pointer, C provides a shorthand for it: the arrow operator, written `->`, with no space between the two characters.

```
cPtr->x = 10.5;
```

This means exactly the same thing as `(*cPtr).x = 10.5;`. Whenever you see `->`, read it as “dereference this pointer, then take this member.” All three of the following lines do the same thing:

```
c.x = 10.5;  
(*cPtr).x = 10.5;  
cPtr->x = 10.5;
```

The rule for choosing between the dot and the arrow is simple: if what you have is the structure itself, use the dot; if what you have is a pointer to the structure, use the arrow.

Structures and functions

Structures can be passed to functions and returned from them, just like any other type. The one thing that trips people up is the type name itself: `struct coord` is one type, made of two words, and both words have to appear every time you write it, including in a function’s return type and parameter list.

```
struct coord addCoordinates(struct coord point1, struct coord point2);
```

Read this the same way you would read `double addNumbers(double a, double b);`. We have a function called `addCoordinates`, taking two arguments of type `struct coord`, returning a value of type `struct coord`. There is no such thing as a function “of type `struct`” on its own. `struct` is only the keyword that introduces a structure tag; the type is `struct coord`, both

words together, and you cannot drop either one (except if you use a little hack that we will get to in a moment).

Here is the whole program:

```
/*
 * Adding coordinates.
 */

#include <stdio.h>

struct coord
{
    double x;
    double y;
};

struct coord addCoordinates(struct coord point1, struct coord point2);

int main()
{
    struct coord pointA = {10.0, 11.0};
    struct coord pointB = {1.0, 2.0};
    struct coord pointSum;

    printf("Point A is at: x = %f, y = %f\n", pointA.x, pointA.y);
    printf("Point B is at: x = %f, y = %f\n", pointB.x, pointB.y);

    pointSum = addCoordinates(pointA, pointB);

    printf("Point Sum is at: x = %f, y = %f\n", pointSum.x, pointSum.y);

    return 0;
}

struct coord addCoordinates(struct coord point1, struct coord point2)
{
    struct coord result;

    result.x = point1.x + point2.x;
    result.y = point1.y + point2.y;

    return result;
}
```

which prints:

```
Point A is at: x = 10.000000, y = 11.000000
Point B is at: x = 1.000000, y = 2.000000
Point Sum is at: x = 11.000000, y = 13.000000
```

Walk through what actually happens when `addCoordinates(pointA, pointB)` is called. `pointA` and `pointB` live in `main`, each a pair of boxes holding an `x` and a `y`. Calling the function does not hand over `pointA` and `pointB` themselves; exactly as with a `double` or an `int` argument, C makes fresh copies. Two new structures spring into existence inside `addCoordinates`, called `point1` and `point2` there, and the values of `pointA` are copied into `point1`'s boxes while the values of `pointB` are copied into `point2`'s boxes. Inside the function, a third structure, `result`, is created, and its members are filled in from `point1` and `point2`. When the function returns, the two boxes that make up `result` are copied out and become the value of the call, which is then copied again into `pointSum` back in `main`. Once the function ends, `point1`, `point2` and `result` are gone; only the copy that made it into `pointSum` survives.

This is call-by-value applied to a structure rather than a single number: nothing that happens inside `addCoordinates` can reach back and change `pointA` or `pointB`, because the function never had access to them in the first place, only to copies.

Shorter names for your own types with `typedef`

There is one thing about structures that is undeniably clunky. Declaring an ordinary variable is one word:

```
int a;
```

but declaring a structure variable is two:

```
struct coord a;
```

and you have to remember, every single time, that it is `struct coord` and not just `coord`. What we actually want is to write:

```
Coordinate a;
```

and have that mean the same thing. The tool for this is `typedef`, a keyword that creates an alias, a second name, for a type that already exists. `typedef` does not create a new type; it only gives an existing one a shorter name to answer to.

There are two equivalent ways to use it. The first defines the structure as usual and then aliases it on a separate line:

```
struct coord
{
    double x;
    double y;
};
```

```
typedef struct coord Point;
```

The second folds the two steps into one, by writing `typedef struct` and giving the name right where the closing brace would otherwise end the definition:

```
typedef struct
{
    double x;
    double y;
} Point;
```

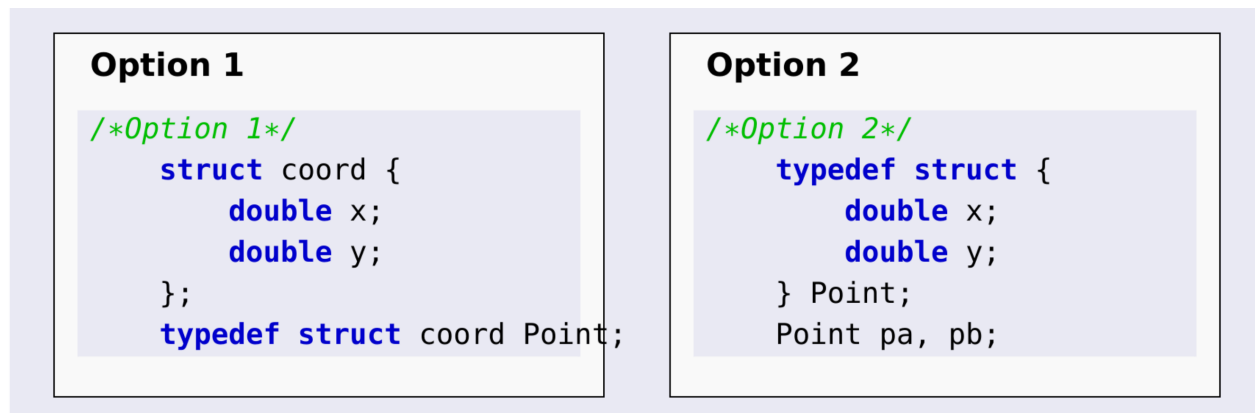


Figure 2: Two equivalent ways to write the same typedef. Option 1 defines `struct coord` first and aliases it afterwards; Option 2 folds the alias into the structure definition itself, and does not need a separate tag at all.

Either way, `Point` now means `struct coord`, and from here on you can declare variables the short way:

```
Point pa, pb;
```

instead of:

```
struct coord pa, pb;
```

By convention, a type created with `typedef` is often given a name starting with a capital letter, `Point` rather than `point`, so that at a glance you can tell a type name from a variable name.

Rewriting the coordinate-adding program with `typedef` makes the function prototype noticeably easier to read:

```

/*
 * Adding coordinates.
 */

#include <stdio.h>

typedef struct
{
    double x;
    double y;
} Point;

Point addCoordinates(Point point1, Point point2);

int main()
{
    Point pointA = {10.0, 11.0};
    Point pointB = {1.0, 2.0};
    Point pointSum;

    printf("Point A is at: x = %f, y = %f\n", pointA.x, pointA.y);
    printf("Point B is at: x = %f, y = %f\n", pointB.x, pointB.y);

    pointSum = addCoordinates(pointA, pointB);

    printf("Point Sum is at: x = %f, y = %f\n", pointSum.x, pointSum.y);

    return 0;
}

Point addCoordinates(Point point1, Point point2)
{
    Point result;

    result.x = point1.x + point2.x;
    result.y = point1.y + point2.y;

    return result;
}

```

Point addCoordinates(Point point1, Point point2) now reads almost like ordinary English: a function that takes two points and returns a point. That is exactly what typedef buys you. It is worth noting that typedef is not limited to structures either; you can use it to give a

shorter name to any type, a pointer type included, though structures are by far the most common use.

Looking forward

Structures give you a way to name a group of related variables and treat it as one thing. That one idea turns out to be the foundation for almost everything left in this course: arrays of structures, structures allocated with `malloc`, and structures containing pointers to other structures of the same type, which is how linked lists, stacks, queues and trees are all built. None of that is new machinery. It is `struct`, the dot, the arrow and `typedef`, combined with the pointers and arrays you already know.

Exercises

1. Define `struct vector` with members `i`, `j` and `k`, declare a variable `v` initialised to `{1.0, 2.0, 3.0}`, and print `sizeof(v)`. Explain the number you get in terms of `sizeof(double)`.
2. Write a structure `struct rectangle` with `double width` and `double height` members, and a function `double area(struct rectangle r)` that returns its area. Write a main that declares two rectangles, prints both their areas, and prints the area of whichever one is larger.

3. Given

```
struct coord c;  
struct coord *cPtr = &c;
```

write the same assignment to `c.x` three different ways: using `.`, using `(*cPtr).`, and using `->`. Confirm with `printf` that all three leave `c.x` with the same value.

4. Write a `struct student` with a `char name[30]`, an `int age`, and a `struct student *bestFriend`. Declare two students, point the first one's `bestFriend` at the second, and print the second student's name by following the pointer from the first, using `->`.
5. Take the `addCoordinates` example and write a second version, `addCoordinatesByRef`, that takes three pointers to `struct coord`, two inputs and one output, and writes the sum into the output through the pointer instead of returning it. Call both versions from main on the same two points and confirm they agree.
6. Use `typedef` to give `struct student` from exercise 5 the shorter name `Student`, in both of the two forms shown in this lecture. Rewrite your exercise 5 program using `Student` instead of `struct student` throughout.
7. `sizeof` a structure is not always simply the sum of `sizeof` on each of its members; compilers are allowed to insert padding between members for performance reasons. Write a `struct pair` with a `char` member followed by a `double` member, print `sizeof(char)`, `sizeof(double)` and `sizeof(struct pair)`, and see whether the last one is what you would naively expect. (If it is not, that is padding, not a bug.)